CampusPulse

Campus Event Discovery and Registration Portal

Interview explanation guide

This guide is based on the current repository implementation. It gives you a short introduction, an architecture diagram, user flows, API and data-model details, technical trade-offs, security limitations, demo steps, and answers to common interviewer questions.

Important: the implementation and the recommended production improvements are deliberately separated, so you can explain the project accurately.

1. Your 60-90 second introduction

CampusPulse is a full-stack campus event discovery and registration portal. It solves the problem of scattered event information and manual attendance tracking by giving students one place to discover events, filter them by category or department, register for a seat, and receive a digital event pass. Organizers get a separate dashboard to create and manage events, monitor capacity, view attendee rosters, export CSV data, and mark attendees as checked in.

The frontend is a React 18 single-page application built with Vite and Tailwind CSS. It communicates with an Express REST API using JSON requests. The backend is separated into routes, controllers, authentication middleware, Mongoose models, and a database abstraction. MongoDB Atlas is the primary store, while a file-backed JSON store makes local development possible when MongoDB is not configured.

Authentication uses bcryptjs for password hashing and JWTs for sessions. The application has student and admin roles. The important business rules are kept on the backend: event existence, capacity checks, duplicate-registration checks, pass-code creation, registration-count updates, and seat restoration when a registration is cancelled.

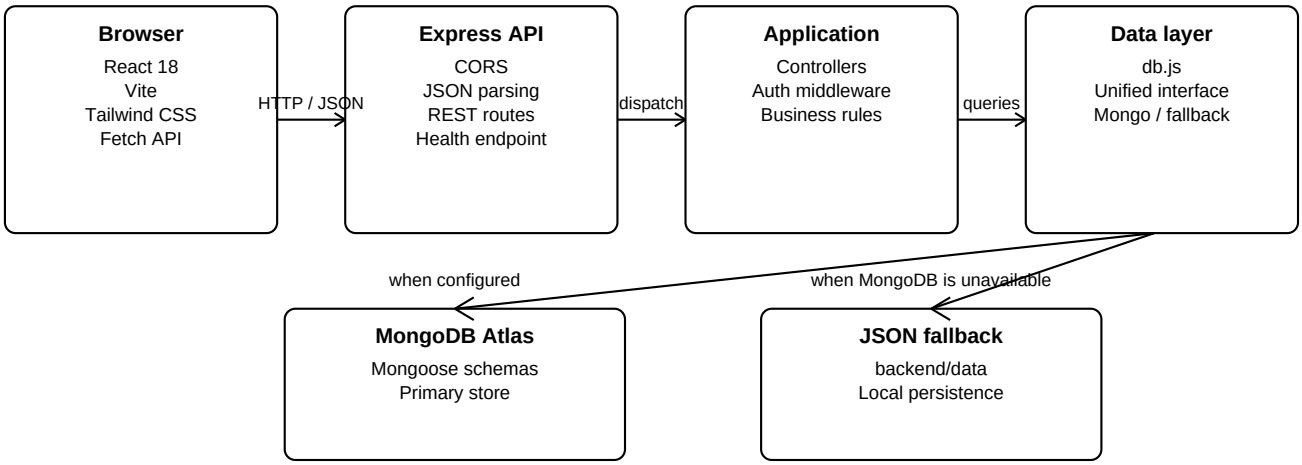
A limitation I would mention honestly is that this is a portfolio-scale implementation. Before production, I would enforce authorization on every admin endpoint, make capacity reservation atomic under concurrency, and replace the QR-style display with a real signed and verifiable QR token.

Suggested transition: **"I will first explain the architecture, then demonstrate the student registration flow and the organizer check-in flow."**

2. Project at a glance

Area	Current implementation
Product	CampusPulse: event discovery, registration, digital pass display, organizer operations
Frontend	React 18, Vite, Tailwind CSS, lucide-react; code is in frontend/src
Backend	Node.js, Express REST API; code is in backend
Persistence	MongoDB Atlas through Mongoose, with backend/data/db_store.json fallback
Authentication	Signup/login, bcryptjs password hashing, JWT generation and verification helpers
Roles	Student and admin/organizer experiences
Development ports	Frontend 3000; backend 5000; Vite proxies /api to the backend

3. Architecture diagram



How to explain it

- The browser runs the React SPA. App.jsx owns session state, role-aware navigation, theme state, logout, and toast notifications. Feature components own their local loading, modal, and request state.
- Express is the API boundary. server.js configures CORS and JSON parsing, mounts the route modules, exposes /api/health, and can serve frontend/dist after a production build.
- Routes map HTTP methods and URLs to controller functions. Controllers validate basic inputs, call the data layer, and return consistent JSON responses.
- db.js exposes one interface for events, registrations, statistics, and users. It hides whether the active persistence mechanism is MongoDB Atlas or the JSON fallback.
- The data layer is where the key workflow rules are implemented: capacity checks, duplicate checks, pass-code generation, count updates, and registration deletion.

4. Main user journeys

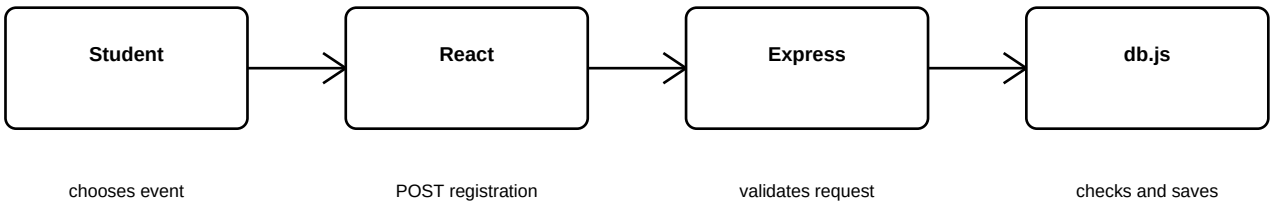
Student journey

- The student signs up or signs in through LandingPortal or AuthModal. The server returns a JWT and a safe user summary; the frontend stores them in sessionStorage.
- EventDiscovery calls GET /api/events. Search and filters become query parameters such as search, category, department, dateFilter, and openOnly.
- EventModal displays event details and capacity, then sends POST /api/registrations with the event ID and student information.
- The backend confirms the event exists and has capacity, creates the registration, generates an EVT-xxxxxx pass code, increments registeredCount, and returns the pass.
- MyRegistrations loads passes by student ID. Cancelling a pass sends DELETE /api/registrations/:id and restores one seat.

Organizer journey

- AdminDashboard loads events and statistics in parallel. It shows total events, registrations, capacity utilization, and check-in rate.
- The organizer can create, edit, publish, cancel, complete, or delete events; view an attendee roster; export CSV; and toggle check-in.

5. Registration flow



success: create EVT-xxxxxx pass code, increment registeredCount, return pass

What happens on the server

- registrationController.createRegistration validates the required event ID, student name, email, and student ID.
- db.registrations.create loads the event and rejects missing events or a full event.
- The MongoDB path checks for an existing registration by event plus email or student ID. The registration receives a pass code and a snapshot of event information.
- The event's registeredCount is incremented. The client then shows TicketPassModal and refreshes the event list so the capacity display is current.

6. Frontend structure

Component	Responsibility	Relevant behavior
App.jsx	Application shell and session state	Role-aware tabs, guest mode, theme toggle, logout, toasts
LandingPortal / AuthModal	Authentication UI	POST login and signup requests
EventDiscovery	Discovery and filtering	GET events with URLSearchParams; cards show remaining seats
EventModal	Event details and registration	Capacity display and POST registration
MyRegistrations	Student passes	Pass lookup, print/display, cancellation
TicketPassModal	Pass presentation	Pass code, event details, print action
AdminDashboard	Organizer operations	CRUD, metrics, roster, check-in, CSV export

Good frontend points to say

- The app uses component-level state because each feature has a clear boundary and the current size does not require a global state library.
- After mutations, the relevant components refetch data so capacity counters, dashboard statistics, and rosters are not stale.
- The Vite proxy lets frontend code use relative /api URLs during development instead of hard-coding the backend origin in every component.
- Loading, empty, error, confirmation, and success states are represented in the UI.

7. Backend structure

Layer	Purpose	Files
server.js	Express setup, middleware, routes, health check, static build, startup and seeding	backend/server.js
Routes	Map methods and URLs to handlers	backend/routes/*.js
Controllers	Input checks and JSON responses	backend/controllers/*.js
Middleware	JWT verification and role helper	backend/middleware/authMiddleware.js
Models	Mongoose validation and indexes	backend/models/User.js, Event.js, Registration.js
Data abstraction	MongoDB/fallback switch and core business rules	backend/db.js
Seed	Initial events and sample registrations	backend/seed.js

8. API surface

Method	Endpoint	Purpose
POST	/api/auth/signup	Create student or admin account; admin signup checks a configured passcode
POST	/api/auth/login	Authenticate by email, name, or student ID and return a JWT
GET	/api/auth/me	Return the authenticated user through protect middleware
GET	/api/events	List events with category, department, search, date and open-seat filters
GET	/api/events/:id	Get one event
POST / PUT / DELETE	/api/events[/:id]	Create, update, or delete an event
GET	/api/events/stats	Return dashboard totals and utilization metrics
POST	/api/registrations	Register a student and issue a pass
GET	/api/registrations/user/:studentId	Return a student's registrations
GET	/api/registrations/event/:eventId	Return an event roster
PUT / DELETE	/api/registrations/:id[/checkin]	Toggle check-in or cancel registration

9. Data model and business rules

The project uses three main MongoDB collections. Registration records copy key event fields so a pass can be displayed without depending on a second event lookup. That is useful for a historical snapshot, although event edits will not automatically update existing snapshots.

Collection	Important fields	Rules and relationships
User	name, email, password, role, department, studentId	Email is unique; role is student/admin; password is hidden from normal reads and hashed before save
Event	title, category, department, date, time, location, capacity, registeredCount, status, tags, agenda	Enums validate category, department, and status; capacity has a minimum of 1
Registration	eventId, event snapshot, student info, passCode, checkedIn, timestamps	eventId references Event; MongoDB compound indexes prevent duplicate event/student registrations

Rules worth explaining

- Capacity: registration is rejected when registeredCount is at least capacity. The Mongo update path changes a published event to Sold Out when it reaches capacity.
- Duplicate prevention: the Mongo path checks student email and student ID for the same event, and Registration.js declares compound unique indexes.
- Cancellation: removing a registration decrements registeredCount if it is above zero, restoring availability.
- Seeding: on startup, seedData adds the initial event catalogue if the event collection is empty and attempts sample registrations for the seeded hackathon.

10. Security and production-readiness

Implemented now	Improvement to discuss
bcryptjs password hashing and password exclusion in the Mongoose user model	Require a production JWT secret and fail startup if it is missing; do not use a fallback secret in production
JWT creation plus protect and authorize helpers	Apply protect and authorize('admin') to event mutations, roster, check-in, and other organizer-only routes; do not rely on client-side role gating
Admin signup passcode from environment configuration	Use an invitation or managed operator provisioning flow; never expose default credentials in the UI
Mongoose validation and unique indexes	Use atomic conditional seat updates or transactions to prevent overselling during concurrent requests
CORS and JSON parsing	Allowlist origins, use HTTPS, add rate limiting and secure headers, validate request schemas, and avoid exposing raw errors
Pass code and printable QR-style visual	Generate a signed, real QR payload and validate it server-side at entry

11. Technical decisions and trade-offs

Why React and Vite?

The portal is interactive but still a focused single application, so a component-based SPA is a good fit. Vite gives fast development feedback and a simple production build. Local component state keeps the current code understandable without adding global state-management overhead.

Why Express REST?

The API gives one place for validation, authorization, business rules, and persistence. REST also leaves room for a future mobile app, kiosk, or check-in client to use the same event APIs.

Why a database abstraction?

db.js means controllers do not need to know whether data is in MongoDB or the local JSON fallback. The trade-off is additional complexity and the responsibility to keep both implementations behaviorally consistent.

Why denormalize event fields in a registration?

A registration is also a user-facing pass. Storing event title, date, time, and location makes the pass fast to render and preserves what the student registered for. The trade-off is that later event edits do not automatically rewrite old snapshots.

12. Interview questions and answers

Question	Answer outline
How do you prevent overbooking?	The backend checks capacity before creating a registration and increments registeredCount. For production, I would use an atomic conditional update or transaction so concurrent requests cannot oversell.
How does role-based access work?	The JWT contains the user ID and role. protect verifies the token and loads the user; authorize checks allowed roles. I would apply these middleware functions to all admin routes before production.
What if MongoDB is unavailable?	connectDB leaves the Mongo flag false and db.js uses a JSON file-backed store. This helps local demos, but it is not a replacement for a shared database in a multi-instance deployment.
Why sessionStorage?	It provides a tab-scoped demo session and supports separate tabs. The trade-off is XSS exposure; production could use secure httpOnly cookies with CSRF protection.
How would you test it?	Add API integration tests for auth, capacity boundaries, duplicates, cancellation, authorization, and check-in; component tests for filters and states; and an end-to-end student-to-admin workflow.
What was challenging?	Keeping event capacity, registration records, and UI refreshes consistent across MongoDB and the fallback store. The unified data interface and explicit refreshes address that.
How would you scale it?	Use managed MongoDB and indexes, pagination, caching for discovery, atomic updates, background notifications, observability, and stateless API instances behind a load balancer.

13. Demo checklist

Step	Show	What to say
1	Landing page and role selector	The same product supports student and organizer workflows with role-aware navigation.
2	Event search and filters	Filters become API query parameters, so the server/data layer remains the source of the listing.
3	Event details and capacity	The student sees remaining seats before registering; the server still rechecks capacity.
4	Register and display pass	The backend creates the record, generates a pass code, updates capacity, and returns the pass.
5	My Passes and cancellation	The student can retrieve registration history and restore a seat by cancelling.
6	Organizer dashboard	The organizer can manage event lifecycle, view KPIs, and open a roster.
7	Check-in and CSV	Check-in is persisted and the roster can be exported for operations.
8	Limitations	Mention server authorization, atomic capacity, automated tests, and real QR validation as next steps.

14. How to run it

Do not share backend/.env values in an interview. Explain that database connection strings, JWT secrets, and admin configuration are environment variables.

Backend

```
node backend/server.js
```

Frontend development server

```
npm --prefix frontend run dev
```

Production frontend build

```
npm run build
```

Useful endpoint: GET /api/health confirms that the REST API is running. The frontend dev server uses port 3000 and proxies /api to port 5000.

15. Code map for follow-up questions

If asked about	Open this file	Why
Server setup	backend/server.js	Express setup, routes, health, static build, startup
Storage and capacity	backend/db.js	Mongo/fallback switch, filtering, registration rules, stats
Authentication	backend/controllers/authController.js and backend/middleware/authMiddleware.js	JWT creation, password matching, token verification, role helper
Data design	backend/models/User.js, Event.js, Registration.js	Validation, indexes, timestamps, relationships
Student experience	frontend/src/components/EventDiscovery.jsx and EventModal.jsx	Filtering, capacity display, registration request
Organizer experience	frontend/src/components/AdminDashboard.jsx	CRUD, metrics, roster, check-in, CSV
Development setup	frontend/vite.config.js and package.json files	Ports, proxy, scripts, dependencies

Final answer to close with

The project demonstrates an end-to-end workflow rather than only a UI: role-aware authentication, event search, capacity-aware registration, digital pass issuance, organizer analytics, roster management, check-in, and export. The architecture separates UI, HTTP handling, business rules, and persistence. My next priorities would be server-enforced authorization, atomic seat reservation, automated tests, real QR verification, and observability.